TECHNICAL REFERENCE · FOR FRONTEND DESIGN & IMPLEMENTATION

Mend Sure

Complete backend architecture, database schema, and API reference

Medical tourism platform connecting patients from the US/UK with affordable treatment in India · Prepared August 28, 2026

1. Project Overview

Mend Sure is a medical tourism platform. It connects patients from the US, UK, and other countries — people who can't afford treatment costs at home — with accredited hospitals and doctors in India, at a fraction of the price. The core user journeys are:

1. **Discovery** — browsing treatments, hospitals, and doctors, with transparent India-vs-US/UK cost comparisons.
2. **Lead capture** — a "Get a Free Quote" enquiry form (the main conversion event of the business).
3. **Patient account** — signup/login, uploading medical reports for review, and (new) seeing their own past enquiries.

The founder's explicit priorities: keep the codebase simple and human-readable (no unnecessary abstraction layers, no TypeScript), and avoid fabricating real hospital partnerships, accreditations, or patient testimonials — all seed/demo content is clearly marked as placeholder.

2. Tech Stack

LAYER	CHOICE	NOTES
Frontend	React 19 + Vite 8	Plain JavaScript, no TypeScript
Styling	Tailwind CSS v4	CSS-first theme (`@theme` block), no `tailwind.config.js`
Routing	React Router v7	`BrowserRouter`, standard declarative routes
Backend	Node.js + Express 4	One router file per resource, no controller/service layers
Database	Supabase (Postgres)	Accessed only from the server, via the service-role key
Auth	Supabase Auth	Email + password, JWT-based sessions
File storage	Supabase Storage	Private bucket for patient-uploaded medical reports
Email	Resend	Optional — new-enquiry notifications; degrades to console logging without a key

> The frontend **never** talks to Supabase directly — it only ever calls the Express API. Express holds the Supabase service-role key and is the only thing that touches the database or storage. Keep this pattern when adding new frontend features.

3. Existing Frontend (already built)

A full marketing/browsing site already exists and is verified working. A redesign or extension should treat this as the starting point, not a blank slate, unless told otherwise.

Pages (`client/src/pages/` , routed in `App.jsx`)

ROUTE	FILE	PURPOSE
`/`	Home.jsx	Hero, stats, About/Mission-Vision tabs, specialty scroll-row, popular treatments, doctors scroll-row, partner hospitals, city badges, "what you get" checklist, testimonials, closing CTA
`/treatments`	Treatments.jsx	List + specialty filter chips (reads `?specialty=` from URL)
`/treatments/:slug`	TreatmentDetail.jsx	Description, cost comparison table, package inclusions, hospitals offering it, enquiry form
`/hospitals`	Hospitals.jsx	List
`/hospitals/:slug`	HospitalDetail.jsx	Description, accreditations, treatments offered there, enquiry form
`/doctors`	Doctors.jsx	List
`/doctors/:slug`	DoctorDetail.jsx	Bio, hospital link, enquiry form
`/how-it-works`	HowItWorks.jsx	5-step patient journey
`/about`	About.jsx	Company blurb
`/testimonials`	Testimonials.jsx	List
`/contact`	Contact.jsx	Main enquiry/quote form
`*`	NotFound.jsx	404

> **Not yet built on the frontend** (backend is ready and waiting): login, signup, forgot/reset password, a profile page, a "My Reports" upload/list/ download UI, and a "My Enquiries" page. See section 10.

Components (`client/src/components/`)

`Navbar` , `Footer` , `Layout` , `Button` (variants: primary/secondary/outline, renders a `<Link>` or a real `<button>`), `TreatmentCard` , `HospitalCard` , `DoctorCard` , `TestimonialCard` , `ConsultationForm` (the reusable enquiry form — includes a honeypot field and a render-timestamp anti-bot check), `CostComparisonTable` , `StatBadge` , `StateMessage` (shared loading/error/empty text), `MissionVisionTabs` , `ScrollRow` (horizontal-scroll carousel with arrow buttons, no external library).

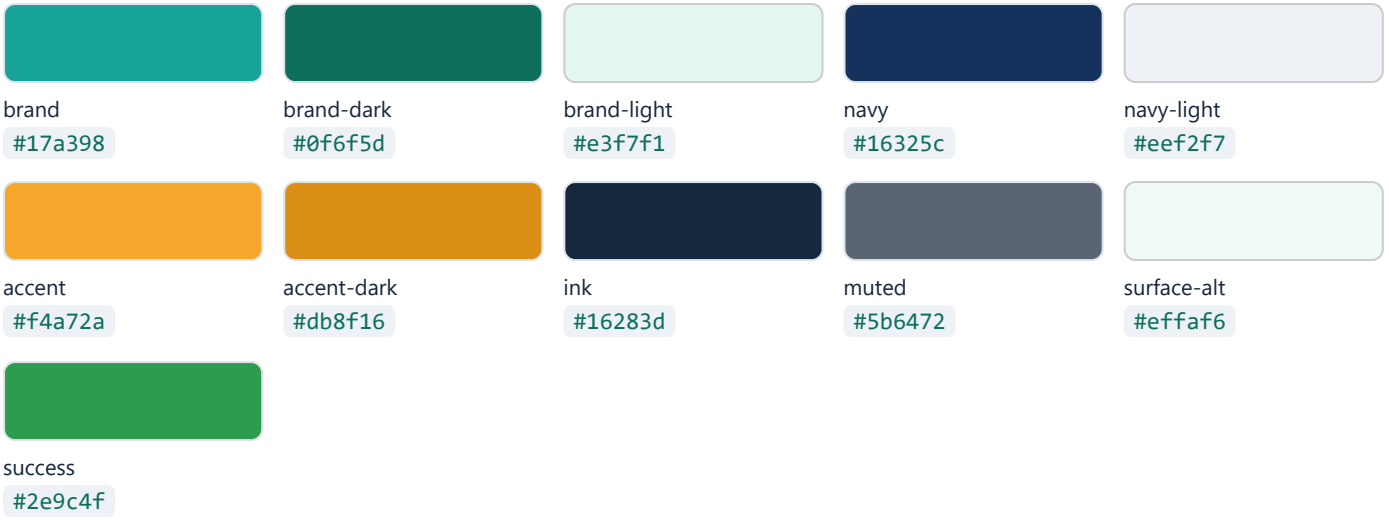
Data fetching (`client/src/lib/`)

```
// useApi.js — shared hook used on every page
const { data, loading, error } = useApi('/treatments')
// Internally: fetch(`/api${path}`), handles abort-on-unmount, loading/error state

// api.js
submitInquiry(formData)  // POST /api/inquiries

// format.js
formatUSD(amount), priceRange(min, max), savingsPercent(indiaPrice, usPrice)
```

In dev, Vite's `vite.config.js` proxies `/api/*` to `http://localhost:5000`, so every frontend call is a same-origin relative path — no CORS handling needed in the frontend code itself.

4. Design System / Brand Tokens

Defined once in `client/src/index.css` as a Tailwind v4 `@theme` block — every variable below is automatically usable as a utility class (e.g. `--color-brand` → `bg-brand`, `text-brand`, `border-brand`). Font is Inter (Google Fonts).



brand `#17a398` | brand-dark `#0f6f5d` | brand-light `#e3f7f1` | navy `#16325c` | navy-light `#eef2f7`

accent `#f4a72a` | accent-dark `#db8f16` | ink `#16283d` | muted `#5b6472` | surface-alt `#effaf6`

success `#2e9c4f`

Logo assets (`client/public/`): `logo-icon.svg` (circular mark only — cross + handshake, used in navbar/footer/favicon), `logo-full.svg` (mark + "MEND SURE — HEALTHCARE SERVICES" wordmark). Both are hand-built SVG, so they scale crisply to any size.

5. Database Schema

Postgres via Supabase. Built across four SQL files, run in order: `schema.sql` → `seed.sql` → `auth-schema.sql` → `schema-updates.sql`.

Public content tables

TABLE	KEY COLUMNS
`treatments`	id, name, slug (unique), specialty, description, price_min_usd, price_max_usd, avg_price_usa_usd, package_inclusions text[], image_url, created_at
`hospitals`	id, name, slug (unique), city, description, accreditations text[], image_url, is_placeholder bool, bed_count int, departments text[], gallery_urls text[], created_at
`hospital_treatments`	id, hospital_id → hospitals, treatment_id → treatments, price_min_usd, price_max_usd (join table: which hospitals offer which treatment, at what price)
`doctors`	id, name, slug (unique), specialty, hospital_id → hospitals (nullable), experience_years, bio, image_url, is_placeholder bool, created_at
`testimonials`	id, patient_name, country, treatment, quote, image_url, is_placeholder bool, created_at
`inquiries`	id, user_id → auth.users (nullable), full_name, email, phone, country, treatment_interested, message, status (default 'new'), source_page, created_at

Patient account tables

TABLE	KEY COLUMNS
`auth.users`	Supabase built-in — id, email, hashed password, etc. Not created by us.
`profiles`	id → auth.users (pk), full_name, phone, country, created_at (auto-created by a trigger the moment someone signs up)
`reports`	id, user_id → auth.users, file_name, storage_path, note, uploaded_at

Storage

Private bucket `patient-reports`. Files live at `{userId}/{uuid}-{originalFilename}`. Never public — always accessed via a short-lived (60s) signed URL from `GET /api/reports/:id/download`.

> **RLS posture:** public content tables have row-level security *enabled with zero policies* (fine — only the server's service-role key touches them, and this means an accidentally-leaked anon key returns nothing rather than everything). `profiles`, `reports`, and the storage bucket have *real* policies scoped to `auth.uid()`, as defense in depth for personal health data.

6. Authentication Model

Login/signup return a Supabase access token + refresh token. The frontend should store these (e.g. `localStorage`) and send the access token on every protected request:

```
Authorization: Bearer <access_token>
```

Access tokens expire after 1 hour — call `POST /api/auth/refresh` with the refresh token to get a new pair rather than forcing re-login. On `POST /api/inquiries`, sending this header is *optional* — it links the enquiry to the account if present, but guests can still submit without it.

7. Complete API Reference

Base path in dev: `http://localhost:5000` (proxied to `/api/*` by Vite). All responses are JSON. Error responses are shaped `{ "error": "message" }`.

Public content

GET `/api/treatments` **public**

Query params: `?specialty=`, `?sort=name|price_min_usd`, `?order=asc|desc`, `?page=`, `?pageSize=` (default 20, max 100). Returns an array; total row count is in the `X-Total-Count` response header.

GET `/api/treatments/:slug` **public**

Returns the treatment plus `hospital_treatments[]`, each with the offering hospital's basic info and that hospital's price for it.

GET `/api/hospitals` **public**

Query params: `?city=`, `?sort=name|city`, `?order=`, `?page=`, `?pageSize=`. `X-Total-Count` header.

GET `/api/hospitals/:slug` **public**

Returns the hospital plus `hospital_treatments[]` (treatments offered there).

GET `/api/doctors` **public**

Query params: `?hospital=` (hospital slug), `?specialty=`, `?sort=name|experience_years`, `?order=`, `?page=`, `?pageSize=`. `X-Total-Count` header.

GET `/api/doctors/:slug` **public**

Returns the doctor plus their `hospitals` object.

GET `/api/testimonials` **public**

Returns all testimonials, newest first.

Enquiry funnel

POST `/api/inquiries` **public (optional auth)**

Body: `{ fullName, email, phone, country, treatmentInterested, message, sourcePage, formLoadedAt }`. `fullName` / `email` / `phone` required. Rate-limited to 5/hour per IP. If an `Authorization` header is present, the row is linked to that account. Returns `{ success: true, inquiry: {...} }`.

GET `/api/inquiries/mine` **requires login**

The logged-in patient's own past enquiries, newest first.

Authentication

ENDPOINT	BODY	RETURNS
POST `/api/auth/signup`	`{ email, password }` (min 8 chars)	`{ access_token, refresh_token, expires_at }`, or a "check your email" message if confirmation is required
POST `/api/auth/login`	`{ email, password }`	`{ access_token, refresh_token, expires_at }`
POST `/api/auth/refresh`	`{ refresh_token }`	New token pair
POST `/api/auth/forgot-password`	`{ email }`	Generic `{ message }` always (no account-enumeration leak)
GET `/api/auth/me` auth	—	`{ email, full_name, phone, country, created_at }`
PATCH `/api/auth/me` auth	`{ fullName, phone, country }`	Updated profile

Signup/login/forgot-password are rate-limited to 10 attempts / 15 min per IP.

Medical report uploads

ENDPOINT	BODY	RETURNS
POST `/api/reports` auth	`multipart/form-data`: field `file` (PDF/JPG/PNG, ≤20MB), optional field `note`	The created report row
GET `/api/reports` auth	—	Array of the caller's own reports: `{ id, file_name, note, uploaded_at }`
GET `/api/reports/:id/download` auth	—	`{ url }` — a signed link valid for 60 seconds
DELETE `/api/reports/:id` auth	—	`{ success: true }`

8. Environment Variables

VARIABLE	WHERE	STATUS
`PORT`	server	Set (5000)
`SUPABASE_URL`	server	Placeholder — needs a real Supabase project
`SUPABASE_SERVICE_ROLE_KEY`	server	Placeholder — needs a real Supabase project
`CLIENT_ORIGIN`	server	Set (http://localhost:5173)
`RESEND_API_KEY`	server	Not set — new-enquiry emails log to console instead until provided
`NOTIFY_TO_EMAIL`	server	Set to a placeholder address, confirm before use
`NOTIFY_FROM_EMAIL`	server	Set to Resend's sandbox sender until a domain is verified

The frontend currently needs no environment variables — it talks to the API via same-origin relative paths through the Vite dev proxy.

9. Deliberately Not Built Yet

These were scoped out on purpose (not overlooked) — confirm before assuming any of them are needed:

- **Staff/admin dashboard** — content (treatments, hospitals, doctors, testimonials) is managed directly in the Supabase table editor, spreadsheet-style.
- **Blog / FAQ** — no content tables or pages exist.
- **Visa-requirements tool** — not built; needs real, country-specific guidance the founder hasn't supplied yet.
- **Public testimonial submission + moderation** — testimonials are currently added only by the founder via Supabase.

10. Suggested Frontend Work

The backend already fully supports the following — only the UI is missing:

1. **Login / Signup pages** — call `/api/auth/login` and `/api/auth/signup`, store the returned tokens.
2. **Forgot / reset password flow** — a form calling `/api/auth/forgot-password`, plus a landing page for the emailed reset link.
3. **My Account / Profile page** — reads `GET /api/auth/me`, edits via `PATCH /api/auth/me`.
4. **My Reports page** — file upload UI (drag/drop or picker) hitting `POST /api/reports`, a list from `GET /api/reports`, a download button per row using `GET /api/reports/:id/download`, and delete.
5. **My Enquiries page** — reads `GET /api/inquiries/mine`, shows each submission's `status`.
6. **Navbar** — needs a logged-in state (avatar/account menu vs. "Log In" link).
7. **Listing pages** — could now use the new `?sort=` / `?order=` /pagination support on Treatments/Hospitals/Doctors, and the new hospital gallery/bed-count/ departments fields on the hospital detail page.

All of this should follow the existing patterns already in the codebase: the `useApi()` hook for GET requests, plain `fetch()` + async/await for mutations (see `ConsultationForm.jsx` and `lib/api.js` for the existing pattern), and the brand tokens in section 4 for styling. No state-management library or TypeScript — plain React state and JavaScript, matching everything already in the repo.